Previews, personajes y reanudación de voz

Informe de causas, implementación, evidencia y pruebas de aceptación.

12 de septiembre de 2026 · Rama `feat/fix-bugs-vimo`

> **Resultado:** implementados los tres bloques solicitados. Compilan el frontend, el sidecar y Rust; se generó el paquete nativo macOS ARM `Vimo.app`. No se instaló, publicó ni reemplazó la aplicación existente.

Problema	Corrección principal
Previews lentas y sin recuperación	Cuatro prompts en un lote, respaldo paralelo y rotación de modelo; cuatro imágenes simultáneas con progreso y publicación individual.
IA deshabilitada en Catálogo	El guion llega al modal y al experto; permite inferir identidad con los campos vacíos.
Reanudar repite narración	Recibos persistentes por solicitud, recuperación de tareas conocidas, partes reutilizables y exclusión de jobs duplicados.

Plan ejecutado

1. Leer contexto, arquitectura y decisiones; seguir cada recorrido desde la UI hasta el proveedor.
2. Separar las correcciones y preservar rutas, orden de etapas y transporte nativo.
3. Implementar y documentar cada cambio inmediatamente en Obsidian.
4. Probar fallos controlados, concurrencia, recuperación y artefactos empaquetados.
5. Cerrar bloques con commits locales y entregar este informe con QA manual.

Commits de corrección inicial: `c7f3779` Catálogo · `eb0f1f4` previews · `249d330` voz · `8a43e44` recuperación del proveedor de respaldo.

> **Alcance de la evidencia:** no se usaron claves reales ni se gastaron créditos. Falta la aceptación con proveedores reales y la comprobación de Windows. Compilar macOS no demuestra funcionamiento de Windows.

1. Previews de Mi estilo

Qué estaba mal

El proceso redactaba cuatro prompts uno por uno, con un modelo grande y esperas/reintentos sobre el mismo proveedor saturado. Después enviaba las cuatro imágenes también de forma serial. Guardaba los resultados juntos al final.

La interfaz ignoraba el avance de prompts y la grilla podía reducirse a la primera tarjeta que recibía un evento. Por eso se percibía poca actividad incluso cuando había trabajo en curso.

Cómo se corrigió

- Política exclusiva de previews: **Llama 4 Scout → GPT OSS 120B Turbo → Ultra**. Cada intento tiene un máximo de 25 segundos; saturación, error de proveedor o respuesta inutilizable activan el siguiente candidato.
- Se comparte la memoria de modelos fallidos entre los pedidos del job. Errores de clave o saldo no rotan modelos: cambiar modelo no arregla la cuenta.
- Por ajuste posterior del usuario: primero una petición con cuatro prompts (3600 tokens). Un lote parcial o inválido conserva lo válido y completa solo faltantes en paralelo (900 tokens cada uno). HTTP/timeout del lote cambia de modelo para el respaldo. Con los cuatro válidos arrancan las cuatro imágenes en paralelo.
- Evolink se consulta cada segundo para estas previews; el resto del pipeline conserva sus intervalos.
- Cada archivo se publica atómicamente al llegar. Su evento incluye ruta y versión temporal para mostrarse antes de que terminen los otros.
- Se mantienen cuatro tarjetas; los resultados parciales y muestras previas no se borran porque otra generación falle.

Qué significa el porcentaje

La redacción combina actividad del stream y estimación temporal. Se limita al 95 % hasta validar el prompt: **no es un porcentaje real del modelo**. Para imágenes se conserva el avance remoto de Evolink y la estimación de Fal. El 100 % de una muestra se emite después de guardarla.

Evidencia

Diecisiete regresiones: rotación, auth/saldo, lote parcial/inválido/duplicado, timeout, cancelación y fallo individual. Se exige un único pedido LLM si el lote funciona; faltantes en paralelo, IDs correctos y ningún POST de imagen con prompts incompletos. Fal/Evolink conservan concurrencia y guardado individual.

> No se promete que las cuatro imágenes terminen exactamente juntas ni descargar en menos de dos segundos: resolución, colas, ancho de banda y CDN siguen influyendo. Se quitaron las esperas seriales y el guardado conjunto de la app.

2. Identidad desde Catálogo

Qué estaba mal

El guion viajaba dentro de `styleContext`, que solo existe para Mi estilo. Al abrir el creador de personajes desde Catálogo, el modal desconocía que había un guion y exigía nombre, edad, rol y descripción completos antes de habilitar la IA.

Cómo se corrigió

Se agregó el guion como dato independiente en `CharacterSection`, `CharacterPickerModal` y `CharacterModal`. Con al menos 20 caracteres, la IA puede inferir la ficha aunque esté vacía. La petición existente `suggest-character-profile` ya aceptaba ese guion: no se creó otro endpoint.

- Los campos que el usuario ya completó conservan prioridad sobre las sugerencias.
- Mi estilo mantiene su experto contextual `design-style-character`.
- La biblioteca sin guion sigue exigiendo los campos previos.
- Se muestran en el modal los errores que propaga la mutación de sugerencia.

> Se conserva la política premium y el fallback determinístico del experto de Catálogo. La corrección habilita y conecta la función; no cambia esa decisión de calidad ni garantiza una latencia equivalente entre modelos.

Verificación de interfaz: 3/3

1. **Catálogo:** se monta la sección real con catálogo vacío y guion; se abre el selector, la pestaña sin foto y la tarjeta de creación. El botón queda habilitado; la petición lleva el guion exacto y los campos se autocompletan.
2. **Biblioteca sin guion:** con campos vacíos, el botón queda deshabilitado.
3. **Mi estilo:** el botón continúa habilitado y envía guion y estilo por su carril original.

Se ejecutó Chrome headless con componentes y hooks React reales, simulando únicamente APIs y disponibilidad de claves. No sustituye una prueba de WebKit/Tauri ni evalúa la calidad de una respuesta real.

Prueba que debés hacer

1. Abrí Crear video → Catálogo y pegá un guion con un protagonista reconocible.
2. Sin seleccionar personajes, abrí Elegir personaje → Sin foto → Crear personaje.
3. Sin completar campos, pulsá Redactar identidad con IA.
4. Confirmá que nombre, rol y descripción correspondan al guion; repetí con otro guion para descartar contexto viejo.
5. Repetí en Mi estilo y sin guion desde Personajes. Sin clave configurada, la IA debe quedar deshabilitada.

3. Reanudación de narración

Qué estaba mal

Se reutilizaban el audio final y las partes terminadas, pero no la solicitud en curso: el identificador remoto solo vivía en memoria. Si se cancelaba antes de guardar el audio, reanudar enviaba otra solicitud. Además, errores de descarga podían activar otro carril y Rust no excluía dos jobs del mismo proyecto.

Cómo se corrigió

- **Antes del POST:** recibo atómico por parte, con estado, hora y huella de texto/voz/motor/modelo/idioma/ajustes/contexto. No contiene claves ni texto.
- **Después de aceptar:** guarda ID remoto y progreso. Si se cancela durante el POST, espera su respuesta para conservar ese ID.
- **Reanudar:** consulta la misma tarea; recupera tiempo/porcentaje y descarga su resultado. Minimax conserva su URL para reintentar la descarga sin repetir la síntesis.
- **Motores síncronos:** terminan y guardan la parte ya enviada; cancelar evita enviar las siguientes.
- **Fallback:** solo ante rechazo confirmado o tarea remota fallida. Un fallo de red, polling, CDN o disco no equivale a permiso para cobrar otra voz. Al reanudar se prioriza el carril registrado, aunque fuera el respaldo; si falta su clave, se bloquea antes de enviar.
- **Audios largos:** reutiliza partes compatibles; FFmpeg escribe un intermedio y solo publica la narración final cuando termina la unión.
- **Concurrencia:** Rust comprueba e inserta el job bajo el mismo mutex. La UI bloquea arranques/reanudaciones duplicados.

Evidencia

Doce casos automatizados verifican número de POSTs, ID/URL persistidos, cancelación, red incierta, cambio de ajustes, error remoto confirmado, recibo corrupto, carril de respaldo y progreso recuperado. El caso de texto largo ejecuta FFmpeg real, fuerza un fallo de unión, reanuda sin nuevos POSTs y decodifica el MP3 completo.

> **Límite de recuperación:** si el proceso muere después de enviar pero antes de recibir ID/URL, no se puede confirmar el resultado automáticamente. Se bloquea el reenvío y se indica revisar el historial remoto; si el audio existe, usarlo como audio propio. Tampoco pueden recuperarse por ID encargos antiguos cuyo identificador nunca se guardó.

El bloqueo nativo cubre esta instancia de app, no otros procesos CLI. Una WebView recargada no reconecta al Channel vivo: debe esperar su terminación y reanudar desde disco. Los recibos pueden contener URLs firmadas; no compartirlos públicamente.

4. Puntos tocados

Rutas relativas al repositorio. Los archivos no listados como código no alteran el comportamiento de producto.

Área	Archivos y responsabilidad
Previews / backend	`pipeline/providers/llm/interactive.ts` — selección/rotación con deadline. `pipeline/commands/styles/generate-style-samples.ts` — concurrencia, progreso y guardado individual. `pipeline/providers/image/evolink.ts`, `pipeline/types/images.ts` — intervalo opcional solo para previews.
Previews / UI	`src/modules/videos/hooks/useStyleSampleGeneration.ts` — progreso de prompts y ruta individual. `src/modules/videos/components/CustomStyleModal.tsx` — cuatro tarjetas estables y visualización incremental.

Personajes	`src/modules/videos/components/CharacterSection.tsx`
	`src/modules/videos/components/CharacterPickerModal.tsx`
	`src/modules/characters/components/CharacterModal.tsx`
	Conexión del guion, habilitación y presentación del error.

Voz / persistencia	`pipeline/providers/tts/request-checkpoint.ts` — recibos atómicos y validación. `pipeline/providers/tts/chunked-synthesis.ts` — recuperación, partes y unión segura. `pipeline/stages/voice/narration.ts` — fallback seguro y tiempo recuperado.
Adaptadores TTS	`pipeline/providers/tts/dubvoice-catalog.ts` — ID y polling recuperables. `pipeline/providers/tts/provider.ts` — contrato interno de checkpoint. `pipeline/providers/tts/{elevenlabs,vbee,fishaudio,minimax}.ts` — propagación del recibo y URL Minimax.
Rust y UI de tarea	`src-tauri/src/pipeline/{commands,jobs,error}.rs` — exclusión por proyecto y error ocupado. `src/modules/videos/hooks/useCreateTask.ts` — guardas de arranque/reanudación. `src/modules/{videos,voices}/constants.ts`, `public/locales/{es,en,pt}.json` — recuperación, reintentos y mensajes.

Pruebas	`scripts/check-preview-regressions.mjs`
	`scripts/check-voice-resume.mjs`
	`scripts/check-packaged-resume.mjs`
	`scripts/check-character-ui.mjs`, `scripts/chrome-session.mjs`

Documentación	`docs/fixes/{catalog-character-context,style-preview-generation,voice-resume}.md` `docs/adr/ADR-0035-recibos-de-voz-y-previews-interactivas.md`, índices ADR y `docs/ARCHITECTURE.md`. Este informe y su fuente en `docs/reports/`; generación PDF en `scripts/build-bugfix-report.mjs`.

En Obsidian: log inmediato de VIMO-TAURI, nota ADR con diagramas Mermaid y APIs compartidas. Se amplió la entrada global existente sobre reintentos que recrean trabajos cobrados. Sin secretos nuevos, cambios de esquema ni dependencias agregadas.

5. Validación y aceptación

Chequeo ejecutado	Resultado
`pnpm format → pnpm lint → pnpm build`	Aprobado
TypeScript del pipeline + `check:locales`	Aprobado; 1183 claves alineadas
Regresiones previews / voz	17/17 + 12/12
Interfaz Chrome, APIs simuladas	3/3 recorridos de personajes
`cargo check` + build Tauri `--bundles app`	Aprobado; macOS ARM
Sidecar dentro de `Vimo.app`	3/3: NDJSON, recibo incierto, cache-hit

Previews: prueba en la app

1. Abrí Mi estilo, elegí un estilo guardado y generá cuatro muestras con Evolink. Repetí con Fal.
2. Verificá cuatro tarjetas desde el inicio, barras durante prompts y arranque conjunto de imágenes en el historial remoto.
3. Una muestra lista debe verse mientras otras siguen pendientes. Si falla una, las ya obtenidas deben conservarse.
4. Compará ID/hora de tareas y visualización. No evalúes concurrencia solo por cuál imagen termina primero.

Voz: prueba en la app

1. Generá una narración corta con ElevenLabs vía DubVoice; durante la voz, navegá a Biblioteca y regresá. Debe continuar.
2. Repetí saliendo para crear otro video. Volvé al primer proyecto y reanudá: debe consultar el mismo ID o usar el audio guardado.
3. Cortá la red después de aceptar la tarea y durante una descarga; restablecela y reanudá. Revisá el historial remoto para confirmar que no se creó otro encargo.
4. Repetí con texto largo, Minimax y ElevenLabs directo. Escuchá las uniones y comprobá duración completa. La parte síncrona enviada puede seguir trabajando después de cancelar.
5. Probá doble clic en Reanudar y cierre forzado durante el POST. Un resultado desconocido debe bloquear reenvío, no inventar éxito.

Ejecutar estas pruebas usa tus proveedores y puede consumir créditos. No se ejecutaron contra tus cuentas durante esta intervención. Repetir en Windows antes de distribuir.

Referencias oficiales para la política de previews

DeepInfra · Llama 4 Scout · GPT OSS Turbo · GPT OSS Ultra · Evolink · Consulta de tareas. Consultadas el 2026-09-12; no implican garantía de latencia.